



The SDLC Lifecycle Mapping:

FinNova — Smart Expense Tracker and Budget Planner

Prepared By:

- *Basaaria Imroz - CSE: AIML(A)*
- *Mehraj Valliya - CSE: AIML(A)*

Institution:

- *Stanley College of Engineering and Technology for Women*

Abstract:

FinNova is a modern *smart expense tracking application* developed to help users manage daily expenses, monitor savings goals, and analyze spending behavior efficiently. The project was created to solve the complexity and usability issues commonly found in traditional financial management applications. Developed using the Lovable AI-assisted development platform, the system emphasizes simplicity, accessibility, responsive design, and financial insight visualization. The project followed the *Software Development Life Cycle (SDLC)* framework to ensure systematic planning, development, testing, optimization, and maintainability. The final application delivers a lightweight, user-friendly, and scalable financial management solution.

1. Introduction

Financial management is a major challenge faced by *students* and *young professionals* who often struggle to balance expenses, savings, and budgeting discipline. Many existing

budgeting applications contain complicated dashboards and unnecessary advanced features, making them difficult for beginners to use consistently.

To address this problem, **FinNova** was developed as a **simple** and **intuitive smart expense tracker** that allows users to:

- Record expenses
- Monitor savings goals
- Analyze spending habits
- Access financial insights visually

The application was developed using Lovable, an **AI-assisted development platform** that accelerated rapid prototyping and modular system development while maintaining clean UI/UX principles.

2. SDLC Methodology Overview

The project followed the **Software Development Life Cycle (SDLC)** framework to ensure **organized** and **systematic development**.

SDLC Model Followed

The project followed a structured iterative SDLC approach consisting of:

1. Planning
2. Requirement Analysis
3. System Design
4. Development & Implementation
5. Testing & Resilience Validation
6. Refactoring & Optimization
7. Deployment & Maintenance

Reason for Selecting SDLC

The SDLC framework was selected because it:

- Improves project organization
- Enhances maintainability

- *Supports systematic testing*
- *Reduces development risks*
- *Improves software quality and scalability*

3. Planning Phase

3.1 Project Goals

The main goals of **FinNova** were:

- *Simplify expense tracking*
- *Encourage savings habits*
- *Provide financial insights visually*
- *Create a beginner-friendly budgeting system*
- *Ensure responsive and stable user experience*

3.2 Feasibility Study

Technical Feasibility

The project was feasible using:

- *Lovable AI-assisted platform*
- *Modern UI development techniques*
- *Modular application structure*

Economic Feasibility

Using Lovable significantly reduced development time and overall implementation complexity.

Operational Feasibility

The system was designed for users with minimal technical expertise, ensuring ease of adoption.

3.3 Project Scope

Included Features

- *Expense tracking*
- *Savings goal management*
- *Spending insights dashboard*
- *User profile management*

Excluded Features

- *Online banking integration*
- *Cryptocurrency tracking*
- *Multi-user synchronization*

3.4 Resource Planning

<u><i>Resource</i></u>	<u><i>Purpose</i></u>
<i>Lovable Platform</i>	<i>Application development</i>
<i>UI Components</i>	<i>Interface creation</i>
<i>Testing Environment</i>	<i>Validation and debugging</i>
<i>Design Assets</i>	<i>Visual interface enhancement</i>

3.5 Timeline Overview

<u>Phase</u>	<u>Activity</u>
<i>Planning</i>	<i>Problem identification and feature planning</i>
<i>Analysis</i>	<i>Requirement gathering</i>
<i>Design</i>	<i>UI/UX and architecture design</i>
<i>Development</i>	<i>Module implementation</i>
<i>Testing</i>	<i>Validation and debugging</i>
<i>Optimization</i>	<i>Refactoring and performance improvements</i>
<i>Deployment</i>	<i>Final preparation</i>

4. Requirement Analysis

4.1 Problem Statement

Many individuals struggle to:

- Monitor daily spending
- Maintain savings discipline
- Understand spending trends
- Access simplified financial insights

Traditional finance applications often *overwhelm users* due to *complexity and poor accessibility*.

FinNova addresses these issues by providing:

- Simple expense management

- *Savings tracking*
- *Spending analysis dashboards*
- *Personalized financial monitoring*

4.2 Functional Requirements

The application should:

- *Allow users to add expenses*
- *Categorize transactions*
- *Set savings goals*
- *Generate financial insights*
- *Manage profile information*

4.3 Non-Functional Requirements

<u><i>Requirement</i></u>	<u><i>Description</i></u>
<i>Performance</i>	<i>Fast and smooth operation</i>
<i>Reliability</i>	<i>Stable application behavior</i>
<i>Usability</i>	<i>Beginner-friendly interface</i>
<i>Maintainability</i>	<i>Modular code structure</i>
<i>Scalability</i>	<i>Easy future expansion</i>
<i>Accessibility</i>	<i>Responsive design support</i>

4.4 Target Users

The application targets:

- *Students*
- *Young professionals*

- *Beginner budget planners*
- *Users seeking lightweight finance management systems*

5. System Design

5.1 System Architecture

The system architecture consists of:

- *User Interface Layer*
- *Expense Management Layer*
- *Savings & Analytics Layer*
- *Profile Management Layer*

This modular structure improves scalability and maintainability.

5.2 UI/UX Design

The interface design focused on:

- *Minimalism*
- *Accessibility*
- *Responsive layouts*
- *Clear navigation flow*
- *Visually engaging indicators*

Modern gradients, icons, and clean typography were used to improve user engagement.

5.3 Module Descriptions

Expense Tracking Module

- *Add expenses*
- *Categorize spending*
- *View transaction history*

Savings Goal Module

- *Create goals*

- *Track progress*
- *Monitor savings milestones*

Insights & Analytics Module

- *Spending visualization*
- *Expense trend analysis*
- *Financial behavior insights*

User Profile Module

- *Manage settings*
- *Edit profile*
- *Customize preferences*

6. Development & Implementation

6.1 Tools & Technologies Used

<u><i>Tool / Technology</i></u>	<u><i>Purpose</i></u>
<i>Lovable</i>	<i>AI-assisted development</i>
<i>Responsive UI Components</i>	<i>Interface design</i>
<i>Modular Architecture</i>	<i>Code organization</i>

6.2 Development Process

The implementation phase involved:

- *Building reusable UI components*
- *Implementing validation logic*
- *Creating responsive layouts*
- *Designing navigation workflows*

- *Structuring modular architecture*

6.3 Coding Structure

The codebase was organized into:

- *UI Components*
- *Business Logic*
- *Validation Logic*
- *Analytics Modules*
- *Profile Management Components*

This improved readability and maintainability.

7. Testing & Resilience

7.1 Input Validation

The system prevents invalid user actions through:

- *Empty field validation*
- *Numeric restrictions*
- *Required form checks*
- *Safe formatting validation*

7.2 Error Handling

The application includes:

- *Null safety checks*
- *Graceful error handling*
- *Controlled fallback mechanisms*
- *Protected navigation routing*

7.3 Edge Case Testing

<u>Test Case</u>	<u>Expected Result</u>	<u>Outcome</u>
<i>Empty expense entry</i>	<i>Validation warning displayed</i>	<i>Passed</i>
<i>Negative transaction amount</i>	<i>Input rejected</i>	<i>Passed</i>
<i>Invalid profile details</i>	<i>Error prompt shown</i>	<i>Passed</i>
<i>Missing saved data</i>	<i>Default values loaded</i>	<i>Passed</i>

7.4 Stability Measures

Additional resilience mechanisms included:

- *Defensive programming*
- *Controlled state management*
- *Responsive UI adaptation*
- *Safe component rendering*

8. Refactoring & Optimization

8.1 Code Improvements

Structural improvements included:

- *Modularized UI components*
- *Feature-based organization*
- *Removal of redundant code*
- *Improved folder hierarchy*

8.2 Performance Optimization

Optimization measures included:

- *Reduced unnecessary UI re-rendering*
- *Improved navigation flow*
- *Streamlined component loading*
- *Cleaned unused assets and variables*

8.3 Readability Improvements

The codebase was improved through:

- *Standardized naming conventions*
- *Simplified logic structures*
- *Added inline documentation*
- *Consistent styling patterns*

8.4 Reusable Components

Reusable modules were introduced for:

- *Form validation*
- *Navigation components*
- *Expense cards*
- *Analytics displays*
- *UI layout consistency*

9. Challenges Faced

Several challenges were encountered during development:

<u>Challenge</u>	<u>Solution</u>
<i>Maintaining responsive UI</i>	<i>Iterative responsive testing</i>
<i>Structuring reusable components</i>	<i>Modular redesign</i>
<i>Preventing invalid financial input</i>	<i>Enhanced validation logic</i>
<i>Balancing simplicity and functionality</i>	<i>UI optimization and feature refinement</i>

10. Deployment / Maintenance

10.1 Deployment Preparation

Before deployment, the application underwent:

- *Final validation testing*
- *UI consistency verification*
- *Navigation flow testing*
- *Performance optimization*
- *Code cleanup*

10.2 Maintenance Strategy

Future maintenance plans include:

- *Bug fixing*
- *Performance monitoring*
- *UI improvements*
- *Feature enhancements*
- *Security updates*

10.3 Future Scalability

The modular architecture allows future expansion including:

- *Cloud synchronization*
- *AI-powered analytics*
- *Multi-device support*
- *Advanced reporting systems*

11. Conclusion

FinNova successfully demonstrates the implementation of the Software Development Life Cycle (SDLC) framework in building a modern smart expense tracking system.

By leveraging Lovable for AI-assisted development, the project achieved:

- *Faster prototyping*
- *Clean and responsive UI design*
- *Improved modularity*
- *Reliable validation handling*
- *Enhanced maintainability*

The final application delivers a lightweight and user-friendly financial management solution while following structured software engineering practices across planning, analysis, design, implementation, testing, optimization, deployment, and maintenance phases.

12. Future Enhancements

Potential future upgrades include:

- *Cloud synchronization*
- *Scanning input from receipts*
- *Multi-device support*
- *Budget recommendation systems*
- *Real-time notifications*
- *Advanced analytics dashboards*
- *Secure authentication integration*

These enhancements would improve scalability, intelligence, and user engagement.

13. References

1. *Software Development Life Cycle (SDLC) Methodologies*
2. *UI/UX Design Principles*
3. *Lovable Development Platform Documentation*
4. *Financial Management System Design Concepts*